Assignment-2

By
1. Bharath Subramaniam Jayakumar
2. Sai Charan Kandepi

Agenda

03 Problem Statement

05 Technical Background

06 Implementation Flow

10 Results

Citation

Gray-Level Co-occurrence Matrix (GLCM) and Texture Features
R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural Features for Image Classification," IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-3, no. 6, pp. 610–621, Nov. 1973, doi: 10.1109/TSMC.1973.4309314.

Texture Analysis Using GLCM (MATLAB Documentation)
MathWorks, "Texture Analysis Using the Gray-Level Co-occurrence Matrix (GLCM)," [Online]. Available: https://www.mathworks.com/help/images/texture-analysis-using-the-gray-level-co-occurrence-matrix-glcm.html. [Accessed: Jun. 09, 2025].

Gray-Level Co-occurrence Matrix Properties (MATLAB Documentation)
MathWorks, "graycoprops - Properties of gray-level co-occurrence matrix (GLCM)," MATLAB Documentation, [Online]. Available: https://www.mathworks.com/help/images/ref/graycoprops.html. [Accessed: Jun 09, 2025].

DBSCAN Clustering Algorithm
M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD'96), Portland, OR, USA, 1996, pp. 226–231

DBSCAN Clustering Algorithm Example (scikit-learn Documentation)
scikit-learn developers, "Demo of DBSCAN clustering algorithm," scikit-learn Documentation, [Online]. Available: https://scikit-learn.org/stable/auto_examples/cluster/plot_dbscan.html. [Accessed: Jun 10, 2025].

Problem Statement

Two major objectives we're focusing on

01

Common Goals

Segment and identify different types of fruits from images using visual features (color, texture, shape). Starting with 3 easily distinguishable fruits, progressively scale to 5 and 10 classes using the Fruits-360 dataset. Evaluate performance using accuracy, recall, and F1-score on test images, including real-world scenarios.

02

Group 9 Problem

Segment and identify fruits using texture features (GLCM and autocorrelation) and cluster them with DBSCAN. Apply morphological operations to refine segmentation. Start with 3 fruit classes and scale to 5 and 10, validating results with accuracy, recall, and F1-score.

Technical Background

GLCM	GLCM (Gray-Level Co-occurrence Matrix
DBSCAN	Density-Based Spatial Clustering of Applications with Noise

Application

- Feature extraction (texture)
- Directional texture analysis
- Contrast, Energy, Homogeneity, Correlation

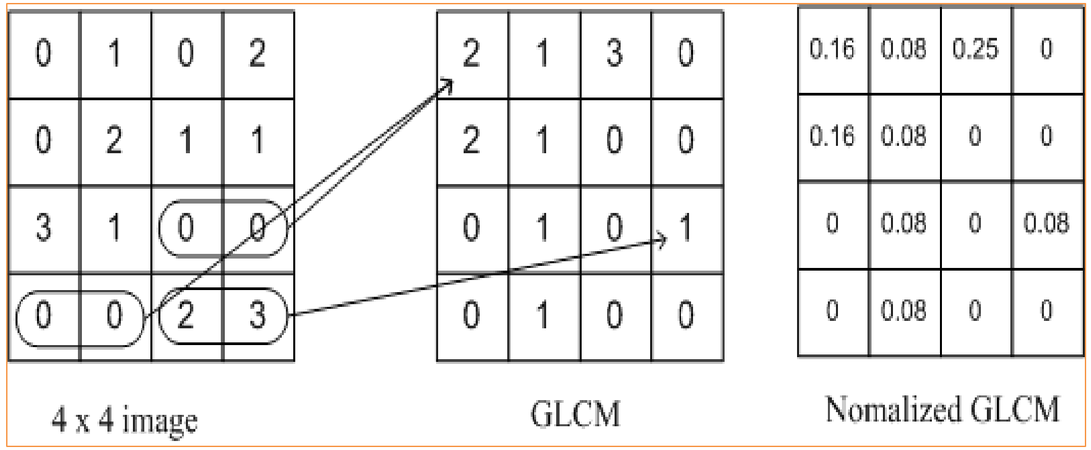
Application

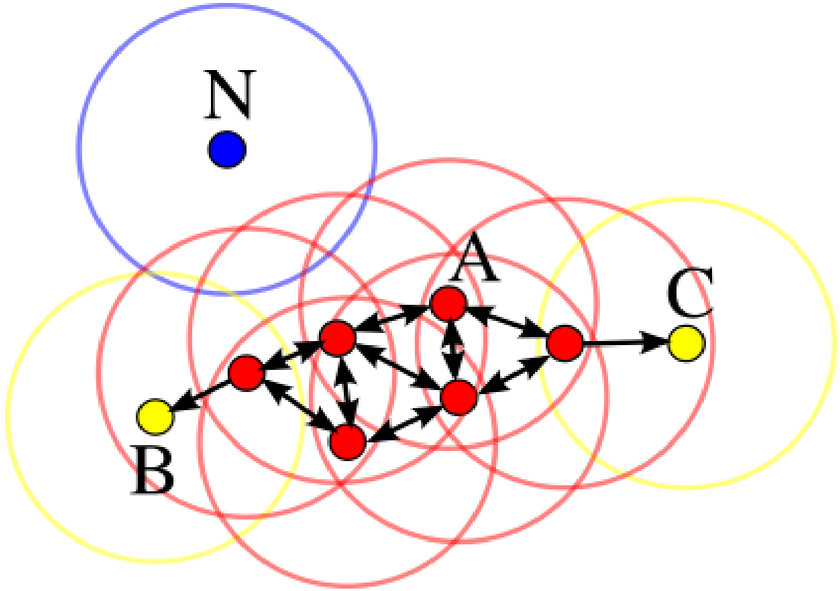
- Clustering algorithm
- Epsilon & Minimum Samples
- Core Point, Border Point, Noise Point

Technical Background

GLCM	GLCM (Gray-Level Co-occurrence Matrix

DBSCAN	Density-Based Spatial Clustering of Applications with Noise





Implementation Flow

Image Processing

Training/Testing/Multi-fruit test Images are loaded

GLCM

Features like contrast, energy, homogeneity, entropy, correlation are calculated

DBSCAN

Points are grouped into clusters based on density, and noise points are labeled as -1

Segmentation

For each contour, the corresponding fruit region is cropped

Visualization

Usage of Confusion matrix and accuracy, recall, and F1-score calculated. With Fruits SegmentationEach fruit is highlighted with a specific color based on its class

Code

- GLCM Features are contrast, energy, homogeneity, entropy, dissimilarity
- HSV Features extracted from image H_mean,S_mean,V_mean,H_std,S_std,V_std
- Contrast = Contrast = $\Sigma\Sigma\ |i-j|^2\ p(i,j)$

  Energy = $\Sigma_{i,j}\ p(i,j)^2$

  Homogeneity = $\Sigma_{i,j}\ p(i,j)\ \log_2(p(i,j))$

  Dissimilarity = $\Sigma_{i,j}\ |i-j|\ p(i,j)$

```python
# ------------------- Logic to Convert Images to Features -------------------

def img2gray(img):
    return cv.cvtColor(img, cv.COLOR_BGR2GRAY)

def compute_glcm_manual(img, d_x=1, d_y=0, levels=8):
    img_scaled = np.clip((img.astype(np.float32) / (256.0 / levels)).astype(int), 0, levels-1)
    glcm = np.zeros((levels, levels), dtype=np.float32)

    for i in range(img_scaled.shape[0]):
        for j in range(img_scaled.shape[1]):
            if 0 <= i + d_y < img_scaled.shape[0] and 0 <= j + d_x < img_scaled.shape[1]:
                glcm[img_scaled[i, j], img_scaled[i + d_y, j + d_x]] += 1

    if np.sum(glcm) > 0:
        glcm /= np.sum(glcm)
    return glcm
#------------------- Feature Extraction -------------------
def extract_glcm_features(glcm):
    contrast = energy = homogeneity = entropy = dissimilarity = 0.0

    for i in range(glcm.shape[0]):
        for j in range(glcm.shape[1]):
            pij = glcm[i, j]
            contrast += (i-j)**2 * pij
            dissimilarity += abs(i-j) * pij
            energy += pij**2
            homogeneity += pij/(1+abs(i-j))
            if pij > 0:
                entropy -= pij * np.log2(pij)

    return [contrast, energy, homogeneity, entropy, dissimilarity]

def extract_enhanced_features(img):
    gray = img2gray(img)
    glcm = compute_glcm_manual(gray)
    texture_features = extract_glcm_features(glcm)

    hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
    h_mean, h_std = np.mean(hsv[:,:,0]), np.std(hsv[:,:,0])
    s_mean, s_std = np.mean(hsv[:,:,1]), np.std(hsv[:,:,1])
    v_mean, v_std = np.mean(hsv[:,:,2]), np.std(hsv[:,:,2])

    return texture_features + [h_mean, h_std, s_mean, s_std, v_mean, v_std]
```

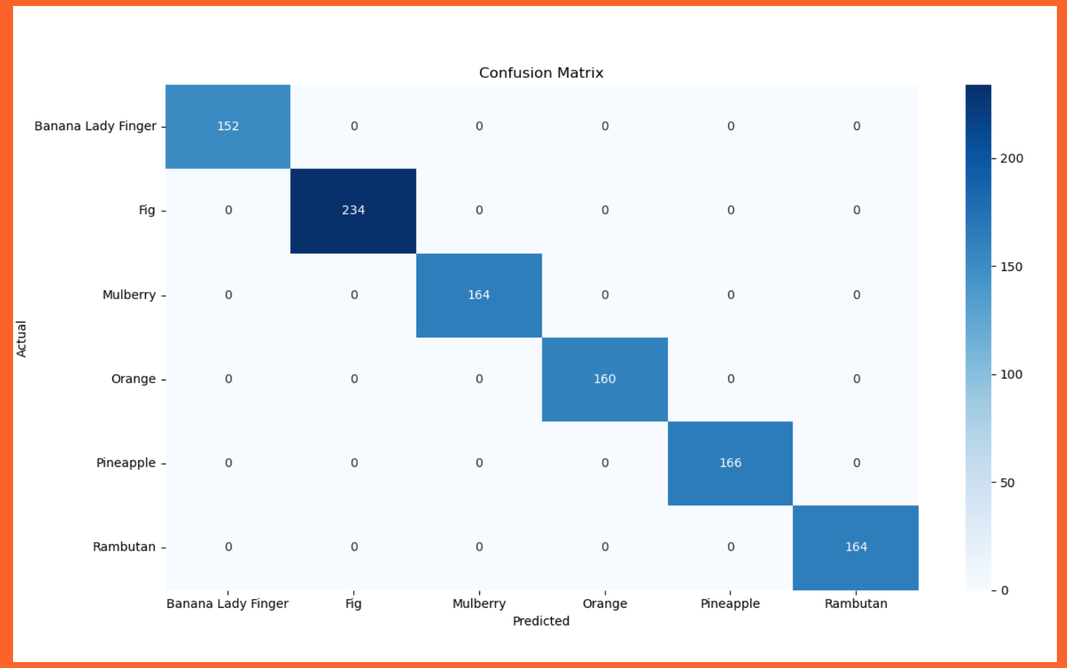
Code

- What is Euclidean Distance, How does it help in our Algorithm
- DBSCAN -Density-Based Spatial Clustering of Applications with Noise :
  - -Initialization
  - -Region Query
  - -Expand Cluster
  - -Iterate Over All Points

```python
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b)**2))
# ------------------- DBSCAN Clustering -------------------
def dbscan(data, eps, min_samples):
    n = len(data)
    labels = [-1] * n
    visited = [False] * n
    cluster_id = 0

    def region_query(idx):
        return [i for i in range(n) if euclidean_distance(data[idx], data[i]) <= eps]

    def expand_cluster(idx, neighbors, cluster_id):
        labels[idx] = cluster_id
        i = 0
        while i < len(neighbors):
            point = neighbors[i]
            if not visited[point]:
                visited[point] = True
                new_neighbors = region_query(point)
                if len(new_neighbors) >= min_samples:
                    neighbors += [nn for nn in new_neighbors if nn not in neighbors]
            if labels[point] == -1:
                labels[point] = cluster_id
            i += 1

    for i in range(n):
        if not visited[i]:
            visited[i] = True
            neighbors = region_query(i)
            if len(neighbors) >= min_samples:
                expand_cluster(i, neighbors, cluster_id)
                cluster_id += 1
    return labels
```

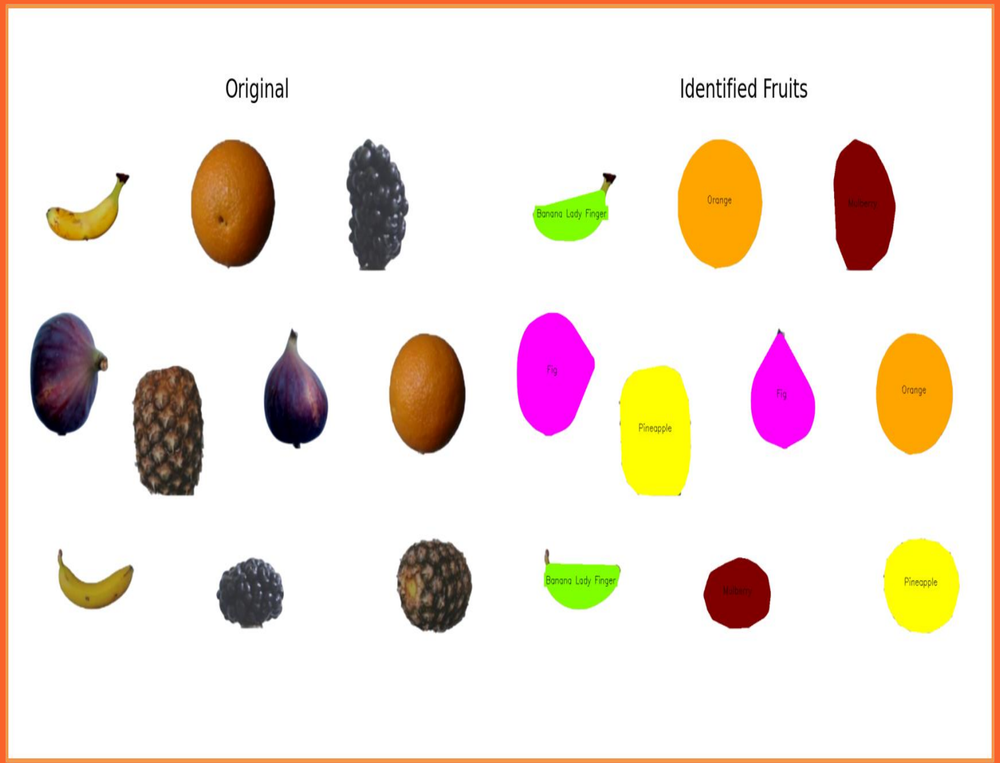
Output / Results

```
Loaded 3105 training and 1040 test images.
Extracting features...

Confusion Matrix:
                    Banana Lady Finger  Fig  Mulberry  Orange  Pineapple  Rambutan
Banana Lady Finger                 152    0         0       0          0         0
Fig                                  0  234         0       0          0         0
Mulberry                             0    0       164       0          0         0
Orange                               0    0         0     160          0         0
Pineapple                            0    0         0       0        166         0
Rambutan                             0    0         0       0          0       164

Classification Report:
                    precision    recall  f1-score   support

Banana Lady Finger       1.00      1.00      1.00       152
               Fig       1.00      1.00      1.00       234
          Mulberry       1.00      1.00      1.00       164
            Orange       1.00      1.00      1.00       160
         Pineapple       1.00      1.00      1.00       166
          Rambutan       1.00      1.00      1.00       164

          accuracy                           1.00      1040
         macro avg       1.00      1.00      1.00      1040
      weighted avg       1.00      1.00      1.00      1040


Accuracy: 100.00%
Test predictions: 1040 assigned, 0 as noise.
```



Identification of Different Fruits

- Banana Lady Finger
- Orange
- Mulberry
- Pineapple
- Fig



Thank You